

List of Entity Methods

Project Title: Coffee POS System

Team Name: POS

Team Members' Names:

Trung Tin Le (Leader)

Bismah Farooq

Minh Tri Chau

Ryan Grubert

Trong Nghia Le

1. Class: Order

1.1. Method: CreateOrder

1.1.1. Description: Creates a new customer order with selected items and initializes its status as "Pending."

1.1.2. Preconditions: Staff must be logged in; menu items must exist; the system must have a database connection; the order must contain at least one item.

1.1.3. Postconditions: The order is stored in the database; the order status is set to "Pending"; the order appears in the active order queue.

1.1.4. Signature: Order CreateOrder(List<OrderItem> items, String createdByStaffIdentifier);

1.2. Method: UpdateOrder

1.2.1. Description: Modifies an existing order's items, quantities, or customizations before payment.

1.2.2. Preconditions: An order must exist; payment must not be processed; staff must be authenticated.

1.2.3. Postconditions: Updated order details saved in the database; order totals recalculated; updated order displayed to the user.

1.2.4. Signature: Order UpdateOrder(String orderIdentifier, List<OrderItem> updatedItems);

1.3. Method: DeleteOrder

1.3.1. Description: Deletes or cancels an existing order before payment processing.

1.3.2. Preconditions: An order must exist; the order must not be completed or paid for; staff must have permission.

1.3.3. Postconditions: Order removed from the database or marked as cancelled; order removed from the active queue.

1.3.4. Signature: void DeleteOrder(String orderIdentifier);

1.4. Method: ViewOrderDetails

1.4.1. Description: Retrieves detailed information about a specific order.

1.4.2. Preconditions: An order must exist; the user must have permission.

1.4.3. Postconditions: Order details (items, status, totals) returned.

1.4.4. Signature: Order ViewOrderDetails(String orderIdentifier);

2. Class: OrderItem

2.1. Method: AddOrderItem

2.1.1. Description: Adds a new item to an existing order.

2.1.2. Preconditions: An order must exist; the menu item must be available; quantity must be positive.

2.1.3. Postconditions: Item added to order; order total updated.

2.1.4. Signature: OrderItem AddOrderItem(String orderIdentifier, String menuItemIdentifier, int quantity);

2.2. Method: UpdateOrderItemQuantity

2.2.1. Description: Updates the quantity of an item within an order.

2.2.2. Preconditions: The order item must exist; quantity must be greater than zero.

2.2.3. Postconditions: Quantity updated; order totals recalculated.

2.2.4. Signature: OrderItem UpdateOrderItemQuantity(String orderItemIdentifier, int quantity);

2.3. Method: RemoveOrderItem

2.3.1. Description: Removes an item from an order.

2.3.2. Preconditions: The order item must exist; the order must not be finalized.

2.3.3. Postconditions: Item removed from order; order totals updated.

2.3.4. Signature: void RemoveOrderItem(String orderItemIdentifier);

2.4. Method: ViewOrderItems

2.4.1. Description: Retrieves all items associated with a specific order.

2.4.2. Preconditions: An order must exist.

2.4.3. Postconditions: List of order items returned.

2.4.4. Signature: List<OrderItem> ViewOrderItems(String orderIdentifier);

3. Class: OrderController

3.1. Method: CreateNewOrder

3.1.1. Description: Handles requests to create a new order and delegates to the Order entity.

3.1.2. Preconditions: Staff must be authenticated; valid order data must be provided.

3.1.3. Postconditions: New order created and persisted; response returned to UI.

3.1.4. Signature: Order CreateNewOrder(List<OrderItem> items);

3.2. Method: ModifyExistingOrder

3.2.1. Description: Handles updates to an existing order.

3.2.2. Preconditions: An order must exist; payment must not be completed.

3.2.3. Postconditions: The order was updated in the database; updated order returned.

3.2.4. Signature: Order ModifyExistingOrder(String orderIdentifier,
List<OrderItem> updatedItems);

3.3. Method: CancelExistingOrder

3.3.1. Description: Cancels an order before payment.

3.3.2. Preconditions: An order must exist and the order must not be completed.

3.3.3. Postconditions: Order marked as cancelled or removed.

3.3.4. Signature: void CancelExistingOrder(String orderIdentifier);

3.4. Method: RetrieveOrderByIdentifier

3.4.1. Description: Retrieves a specific order using its identifier.

3.4.2. Preconditions: Order identifier must be valid.

3.4.3. Postconditions: Order details returned.

3.4.4. Signature: Order RetrieveOrderByIdentifier(String orderIdentifier);

4. Class: OrderStatusController

4.1. Method: UpdateOrderStatus

4.1.1. Description: Updates the status of an order (Pending → In Progress → Completed).

4.1.2. Preconditions: An order must exist and a valid status transition must be provided.

4.1.3. Postconditions: Order status updated in the database; changes reflected in the active queue.

4.1.4. Signature: void UpdateOrderStatus(String orderIdentifier, String status);

4.2. Method: MarkOrderAsInProgress

4.2.1. Description: Sets order status to "In Progress."

4.2.2. Preconditions: An order must exist and the current status must be "Pending."

4.2.3. Postconditions: Status updated to "In Progress."

4.2.4. Signature: void MarkOrderAsInProgress(String orderIdentifier);

4.3. Method: MarkOrderAsCompleted

4.3.1. Description: Sets order status to "Completed."

4.3.2. Preconditions: An order must exist; payment must be completed; the current status must be "In Progress."

4.3.3. Postconditions: Status updated to "Completed"; order removed from active queue.

4.3.4. Signature: void MarkOrderAsCompleted(String orderIdentifier);

5. Class: Payment

5.1. Method: ProcessPayment

5.1.1. Description: Processes a payment for a given order using a selected payment method.

5.1.2. Preconditions: Order exists and is not already paid.

5.1.3. Postconditions: Payment record is created and payment status is updated (success/failure); transaction is logged.

5.1.4. Signature: Payment processPayment(String orderID, String paymentMethod, double amount);

5.2. Method: ValidatePaymentMethod

5.2.1. Description: Validates whether the selected payment method is supported and usable.

5.2.2. Preconditions: Payment method is provided.

5.2.3. Postconditions: Returns true if valid, false otherwise.

5.2.4. Signature: boolean validatePaymentMethod(String paymentMethod);

5.3. Method: ViewPaymentDetails

5.3.1. Description: Retrieves details of a specific payment transaction.

5.3.2. Preconditions: Payment ID exists.

5.3.3. Postconditions: Payment details are returned.

5.3.4. Signature: Payment viewPaymentDetails(String paymentId);

6. Class: Refund

6.1. Method: CreateRefund

6.1.1. Description: Creates a refund for a completed payment.

6.1.2. Preconditions: Payment exists and is completed.

6.1.3. Postconditions: Refund record is created and updated accordingly.

6.1.4. Signature: Refund createRefund(String paymentId, double amount, String refundType);

6.2. Method: ValidateRefundRequest

6.2.1. Description: Validates whether a refund request is eligible.

6.2.2. Preconditions: Payment ID is provided.

6.2.3. Postconditions: Returns true if refund is allowed, false otherwise.

6.2.4. Signature: boolean validateRefundRequest(String paymentId, double amount);

6.3. Method: ViewRefundDetails

6.3.1. Description: Retrieves details of a refund transaction.

6.3.2. Preconditions: Refund ID exists.

6.3.3. Postconditions: Refund details are returned.

6.3.4. Signature: Refund viewRefundDetails(String refundId);

7. Class: TransactionLog

7.1. Method: RecordTransactionEvent

7.1.1. Description: Records a financial transaction (payment or refund) in the log.

7.1.2. Preconditions: Valid transaction data is provided.

7.1.3. Postconditions: Transaction log entry is created.

7.1.4. Signature: void recordTransactionEvent(String orderId, String type, String userId);

7.2. Method: ViewTransactionHistory

7.2.1. Description: Retrieves a list of transaction logs based on filters.

7.2.2. Preconditions: Filter criteria (optional) is provided.

7.2.3. Postconditions: List of transactions is returned.

7.2.4. Signature: List<TransactionLog> viewTransactionHistory(String filter);

8. Class: PaymentController

8.1. Method: ProcessOrderPayment

8.1.1. Description: Coordinates the full payment workflow for an order.

8.1.2. Preconditions: Order exists and is ready for payment.

8.1.3. Postconditions: Payment is processed and recorded; order status is updated.

8.1.4. Signature: boolean processOrderPayment(String orderId, String paymentMethod, double amount);

8.2. Method: StorePaymentRecord

8.2.1. Description: Saves payment details into the system database.

8.2.2. Preconditions: Payment data is valid.

8.2.3. Postconditions: Payment record is persisted.

8.2.4. Signature: void storePaymentRecord(Payment payment);

8.3. Method: VerifyCompletedPayment

8.3.1. Description: Verifies whether a payment was successfully completed.

8.3.2. Preconditions: Payment ID exists.

8.3.3. Postconditions: Returns true if payment is successful.

8.3.4. Signature: boolean verifyCompletedPayment(String paymentId);

9. Class: RefundController

9.1. Method: IssueRefund

9.1.1. Description: Processes and issues a refund to a customer.

9.1.2. Preconditions: Refund request is valid and approved.

9.1.3. Postconditions: Refund is processed and recorded.

9.1.4. Signature: boolean issueRefund(String paymentId, double amount, String refundType);

9.2. Method: ApproveRefundRequest

9.2.1. Description: Approves a refund request after validation.

9.2.2. Preconditions: Refund request exists.

9.2.3. Postconditions: Refund request is marked as approved.

9.2.4. Signature: boolean approveRefundRequest(String refundId);

9.3. Method: RejectRefundRequest

9.3.1. Description: Rejects a refund request.

9.3.2. Preconditions: Refund request exists.

9.3.3. Postconditions: Refund request is marked as rejected.

9.3.4. Signature: boolean rejectRefundRequest(String refundId);

10. Class: TransactionController

10.1. Method: LogFinancialTransaction

10.1.1. Description: Logs payment or refund activities into the transaction system.

10.1.2. Preconditions: Valid transaction data is available.

10.1.3. Postconditions: Transaction is recorded in logs.

10.1.4. Signature: void logFinancialTransaction(String orderId, String type, String userId);

10.2. Method: RetrieveTransactionsByOrder

10.2.1. Description: Retrieves all transactions associated with a specific order.

10.2.2. Preconditions: Order ID exists.

10.2.3. Postconditions: List of related transactions is returned.

10.2.4. Signature: List<TransactionLog> retrieveTransactionsByOrder(String orderId);

11. Class: MenuItem

11.1. Method: AddMenuItem

11.1.1. Description: Adds a new menu item to the Coffee POS system menu.

11.1.2. Preconditions: Administrator must be logged in; required menu item information must be provided; menu item name must be valid.

11.1.3. Postconditions: New menu item is created and stored in the database; menu item becomes available in the menu list.

11.1.4. Signature: void AddMenuItem(String menuItemName, String menuItemCategoryType, String menuItemDescriptionText, double menuItemPriceAmount);

11.2. Method: UpdateMenuItem

11.2.1. Description: Updates the details of an existing menu item such as name, description, category, price, or availability.

11.2.2. Preconditions: Administrator must be logged in; menu item must exist; updated values must be valid.

11.2.3. Postconditions: Menu item information is updated in the database; updated values are reflected in the system.

11.2.4. Signature: void UpdateMenuItem(String menuItemIdentifier, HashMap<String, String> updatedDetails);

11.3. Method: DeleteMenuItem

11.3.1. Description: Removes or deactivates a menu item from the Coffee POS system.

11.3.2. Preconditions: Administrator must be logged in; menu item must exist.

11.3.3. Postconditions: Menu item is removed or marked inactive in the database; item is no longer available to staff users.

11.3.4. Signature: void DeleteMenuItem(String menuItemIdentifier);

11.4. Method: ViewMenuItems

11.4.1. Description: Retrieves and displays the list of menu items available in the system.

11.4.2. Preconditions: User must be logged in; menu data must be available.

11.4.3. Postconditions: Menu item list is returned and displayed to the user.

11.4.4. Signature: List<MenuItem> ViewMenuItems();

11.5. Method: ChangeMenuItemAvailability

11.5.1. Description: Changes the availability status of a menu item.

11.5.2. Preconditions: Administrator must be logged in; menu item must exist; availability value must be valid.

11.5.3. Postconditions: Menu item availability status is updated in the database; item visibility is refreshed in the staff menu.

11.5.4. Signature: void ChangeMenuItemAvailability(String menuItemIdentifier, boolean availabilityStatus);

12. Class: InventoryItem

12.1. Method: UpdateInventoryStock

12.1.1. Description: Updates the current stock quantity of an inventory item.

12.1.2. Preconditions: Administrator must be logged in; inventory item must exist; stock quantity must be zero or greater.

12.1.3. Postconditions: Inventory quantity is updated in the database; new stock level is reflected in the system.

12.1.4. Signature: void UpdateInventoryStock(String inventoryItemIdentifier, int availableQuantityValue);

12.2. Method: CheckInventoryLevel

12.2.1. Description: Checks the current stock level of a specific inventory item.

12.2.2. Preconditions: Inventory item must exist; user must have permission to access inventory data.

12.2.3. Postconditions: Current inventory quantity is returned; no data is modified.

12.2.4. Signature: int CheckInventoryLevel(String inventoryItemIdentifier);

12.3. Method: RestockInventoryItem

12.3.1. Description: Increases the stock quantity of an inventory item after restocking.

12.3.2. Preconditions: Administrator must be logged in; inventory item must exist; restock quantity must be greater than zero.

12.3.3. Postconditions: Inventory quantity is increased in the database; updated stock value is stored in the system.

12.3.4. Signature: void RestockInventoryItem(String inventoryItemIdentifier, int restockQuantityValue);

12.4. Method: ViewInventoryItemDetails

12.4.1. Description: Retrieves detailed information for a specific inventory item.

12.4.2. Preconditions: Inventory item must exist; user must have permission to view inventory details.

12.4.3. Postconditions: Inventory item details are returned and displayed.

12.4.4. Signature: InventoryItem ViewInventoryItemDetails(String inventoryItemIdIdentifier);

13. Class: MenuController

13.1. Method: AddItemToMenu

13.1.1. Description: Handles the request to create and add a new menu item through the controller layer.

13.1.2. Preconditions: Administrator must be authenticated; valid menu item information must be provided.

13.1.3. Postconditions: New menu item is created and persisted in the database; updated menu data is available to the system.

13.1.4. Signature: void AddItemToMenu(MenuItem menuItemDetails);

13.2. Method: EditMenuItemDetails

13.2.1. Description: Handles updates to an existing menu item through the controller layer.

13.2.2. Preconditions: Administrator must be authenticated; menu item must exist; updated details must be valid.

13.2.3. Postconditions: Menu item details are updated successfully in the system.

13.2.4. Signature: void EditMenuItemDetails(String menuItemIdentifier, HashMap<String, String> updatedDetails);

13.3. Method: RemoveItemFromMenu

13.3.1. Description: Handles the removal or deactivation of a menu item from the system.

13.3.2. Preconditions: Administrator must be authenticated; menu item must exist.

13.3.3. Postconditions: Menu item is removed or deactivated in the system database.

13.3.4. Signature: void RemoveItemFromMenu(String menuItemIdentifier);

13.4. Method: RetrieveMenuItems

13.4.1. Description: Retrieves the current list of menu items from the system.

13.4.2. Preconditions: User must be authenticated; menu data must be available.

13.4.3. Postconditions: List of menu items is returned to the requesting interface.

13.4.4. Signature: List<MenuItem> RetrieveMenuItems();

14. Class: InventoryController

14.1. Method: UpdateInventoryQuantity

14.1.1. Description: Handles updates to the stock quantity of an inventory item.

14.1.2. Preconditions: Administrator must be authenticated; inventory item must exist; new quantity value must be valid.

14.1.3. Postconditions: Inventory quantity is updated and stored in the system database.

14.1.4. Signature: void UpdateInventoryQuantity(String inventoryItemIdentifier, int updatedQuantityValue);

14.2. Method: CheckLowStockItems

14.2.1. Description: Retrieves inventory items whose stock levels are below their minimum threshold.

14.2.2. Preconditions: Administrator must be authenticated; inventory data must be available.

14.2.3. Postconditions: List of low-stock inventory items is returned for review.

14.2.4. Signature: List<InventoryItem> CheckLowStockItems();

14.3. Method: GenerateRestockAlert

14.3.1. Description: Generates an alert when an inventory item falls below its configured low-stock threshold.

14.3.2. Preconditions: Inventory item must exist; current quantity must be below the minimum stock threshold.

14.3.3. Postconditions: Restock alert is generated and made visible to the administrator.

14.3.4. Signature: String GenerateRestockAlert(String inventoryItemIdentifier);

15. Class: QueueEntry

15.1. Method: AddOrderToQueue

15.1.1. Description: Adds a new order into the queue for processing.

15.1.2. Preconditions: The order must exist and be valid; the order must not already be in the queue; the queue system must be initialized and active.

15.1.3. Postconditions: A new queue entry is created; the order is added to the queue in the correct position; the queue state is updated.

15.1.4. Signature: void AddOrderToQueue(String orderId);

15.2. Method: UpdateQueueEntryStatus

15.2.1. Description: Updates the status of an existing queue entry.

15.2.2. Preconditions: The queue entry must exist; the provided status must be valid; the system must allow status updates.

15.2.3. Postconditions: The queue entry status is updated; changes are saved in the system; related queue data is refreshed.

15.2.4. Signature: void UpdateQueueEntryStatus(String queueEntryId, String newStatus);

15.3. Method: RemoveOrderFromQueue

15.3.1. Description: Removes an order from the queue when completed or canceled.

15.3.2. Preconditions: The queue entry must exist; the order must currently be in the queue.

15.3.3. Postconditions: The queue entry is removed; remaining entries may be reordered; the queue state is updated.

15.3.4. Signature: void RemoveOrderFromQueue(String queueEntryId);

15.4. Method: ViewQueuePosition

15.4.1. Description: Returns the current position of an order in the queue.

15.4.2. Preconditions: The queue entry must exist; the order must be in the queue.

15.4.3. Postconditions: The position is returned; no system data is modified.

15.4.4. Signature: int ViewQueuePosition(String queueEntryId);

16. Class: SalesReport

16.1. Method: GenerateSalesReport

16.1.1. Description: Generates a sales report for a given date range.

16.1.2. Preconditions: Dates must be valid; start date must be before or equal to end date; sales data must exist.

16.1.3. Postconditions: A report is created with summarized sales data.

16.1.4. Signature: SalesReport GenerateSalesReport(Date startDate, Date endDate);

16.2. Method: CalculateTotalRevenue

16.2.1. Description: Calculates total revenue from sales data.

16.2.2. Preconditions: Sales data must be available; report must contain data.

16.2.3. Postconditions: Total revenue is calculated and returned.

16.2.4. Signature: double CalculateTotalRevenue();

16.3. Method: ViewReportDetails

16.3.1. Description: Returns detailed report information.

16.3.2. Preconditions: Report must exist and be generated.

16.3.3. Postconditions: Report details are returned without modifying data.

16.3.4. Signature: SalesReport ViewReportDetails(String reportId);